



COUNCIL AGENT

開發歷程

從可執行的三階段 CLI 骨架，到可公開測試的信任框架。

起點

v0.1.0

現況

v1.0.0-beta.2

期間

2026.07.15 — 2026.08.12

策略

Tool-First • Spec-driven

五個段落

01	定位與架構	這是什麼、管線如何運作
02	策略與方法	Tool-First 與規格驅動
03	版本演進	v0.1 骨架到 v0.9 安全層
04	清債與信任	v0.9.x 一版一題，再到 Trust Tier
05	現況與下一步	能力、限制、GA 前剩餘工作

閱讀方式

輔助色只標示語意：綠為完成與能力，紅為風險與未完，紫為方法與架構。版面以黑白為主，三色點到為止。

本機 CLI，讓 Agent 依序計劃、執行、校驗

Council Agent 是以 OpenRouter 與 CrewAI 組成的三階段命令列框架。每次推論固定走完計劃、執行、校驗；校驗失敗時由 escalation 修正，並以原成功標準重新校驗。

給誰用 在信任的專案目錄中，需要 Agent 實際改檔、跑測試，又必須留下可審計軌跡的開發者。

做什麼 把 LLM 小隊接到本機工具：讀寫檔案、執行受限指令、跑 pytest，並以證據而非自評做 PASS／FAIL。

不做什麼 不是雲端 Agent 平台，也不是 OS／容器級 sandbox。完整信任框架的 GA 條件仍未全部關閉。

固定四步，失敗不改成功標準



Execution 是 Tool-First 的重心：先讓小隊真的能在工作區操作。Verification 不接受「看起來沒問題」；成功條件若要求 tool/pytest，缺少對應關係、exit code 或成功計數就不得 PASS。

兩組 Preset，全部經 OpenRouter

■ glm-stack

計劃	z-ai/glm-5.2
執行	deepseek/deepseek-v4-flash
校驗	openai/gpt-5.6-luna
升級	openai/gpt-5.6-luna

■ grok-stack

計劃	x-ai/grok-4.5
執行	deepseek/deepseek-v4-flash
校驗	google/gemini-3.5-flash
升級	x-ai/grok-4.5

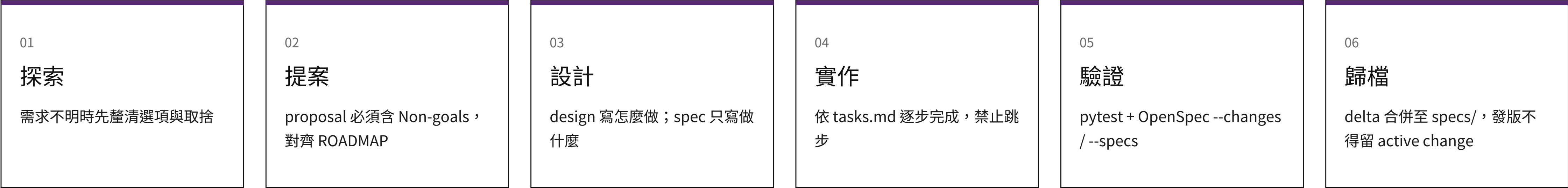
Composer 2.5 為 Cursor 專屬模型，無法經 OpenRouter 呼叫；grok-stack 的執行層因此改用 DeepSeek V4 Flash。角色可換模型，管線順序不變。

Tool-First：先能做，再能拒絕

01	最小改動	沿用 Orchestrator 與三階段管線，在 Execution Crew 掛載 tools，不另起一套執行引擎。
02	邊界先行	檔案與指令操作經 WorkspaceGuard，限制在工作區根目錄；敏感路徑預設拒絕。
03	安全外掛	v0.5 先求功能可用；v0.6 起以分類、確認、審計、政策補強，避免安全設計阻塞主線。
04	證據導向	Verification 逐步改讀 tool 回傳的 exit code、stdout 與結構化測試報告，而不是 LLM 自評。

這條路的代價很清楚：能力會早於完整信任機制出現。文件必須同步寫出限制，而不是把「已能跑」寫成「已安全」。

規格先於程式，OpenSpec 管變更



漸進式整合 函式與單元測試 → CLI／orchestrator 接線 → CrewAI 掛載 → 端對端。每階段 pytest 全綠才能往下。

驗證門檻 提交前必須跑 ./scripts/check.sh。禁止單跑 validate --strict：它會回 Nothing to validate，不代表通過。

ROADMAP 契約 新 change 的 Non-goals 必須對齊該里程碑「不做事」。例如 v0.5 不做 Trust Tier。

一個月，從骨架走到公開測試



- 能力 讓 Agent 在工作區讀寫檔案、跑測試，並把結果交給校驗。
- 控制 為已存在的能力補上分類、確認、審計與專案政策。
- 信任 關閉與 v1.0 衝突的債務，再啟用 Trust Tier 與公開測試。

前兩週把「能做」做完；八月把「能拒絕、能證明、能分級」補齊。中間的停頓不是中斷，而是安全宣稱必須追上能力。

第一天：可跑的三階段骨架

- 專案腳手架 pyproject、uv lock、Git Flow 與 conventional commits。
- 設定與 LLM Pydantic Settings、OpenRouter 工廠、glm-stack／grok-stack YAML preset。
- 三個 Crew Planning 產出計劃；Execution 執行；Verification 給 PASS／FAIL。
- Orchestrator 串起管線與 escalation，校驗失敗時依 max_retries 修正。
- Typer CLI council run 與 council presets list，支援 --verbose。

此版意義

先有一條能跑通的推論管線，後續所有工具、沙箱與安全層都掛在同一條線上，而不是另起執行模型。

工具層、工作區邊界、真實測試

v0.2 Tool 基礎層

- read / write / list / delete
- run_command 回傳 exit code
- 統一 ToolResult，禁止對可預期錯誤 throw
- tools/ 不 import CrewAI，保持可單測

v0.3 Workspace 邊界

- WorkspaceGuard 以 realpath 驗證
- 拒絕路徑穿越與 symlink 逃逸
- 預設拒絕 .env、.git、secrets
- COUNCIL_WORKSPACE_ROOT 可覆寫根目錄

v0.4 測試整合

- run_tests 封裝 pytest
- 結構化 passed / failed / skipped
- Verification 讀取 tool 執行摘要
- max_tool_calls 防止迴圈失控

Sandbox MVP：Agent 真正動手

council sandbox init	在 cwd 建立 .council/，idempotent，不刪既有 session。
council sandbox status	顯示工作區根目錄、已追蹤檔案與最近操作。
Execution 掛載 tools	計劃之後由 Crew 透過 tools 改檔與跑測試。
Session 紀錄	.council/sessions/<id>/ 保存 meta.json 與 tools.jsonl。
--workspace	可覆寫工作區根目錄；未 init 仍可跑管線，但不寫耐久審計。

完成定義（當時）

- 任意 cwd 可 init .council/
- 可建立、修改、刪除工作區檔案
- 可執行 pytest 並取得真實結果
- 路徑穿越與敏感檔案被擋
- Verification 參考測試結果
- 單次 run 有 tool 呼叫上限

安全空窗已在文件標明：僅在信任目錄使用。

能力穩定後，四層安全補強

v0.6	指令分類	read / write / dangerous 的 pattern 啟發式。危險指令預設拒絕或需確認。
v0.7	互動確認	TTY 下 [Y/n]；--yes 供 CI 跳過；無 TTY 預設拒絕。ConfirmMode 仍不是 Trust Tier。
v0.8	審計日誌	結構化 JSONL；council audit show／export。當時仍缺完整性與 redaction。
v0.9	專案政策	council.policy.yaml 允許／拒絕指令與路徑；與預設 denylist 聯集。

到 v0.9.0，產品看起來已有分類、確認、審計與政策。對照 v1.0 DoD 之後才發現：未知指令 fail-open、沒有唯一 dispatcher、政策可被誤讀成授權來源。下一幕是把這些衝突逐項關閉。

2026.08.11：宣稱必須追上實作

發行準備探索對照 ROADMAP、main specs、archive 與程式。判讀原則：任兩者不一致即列衝突；安全宣稱必須同時有拒絕案例、旁路案例與無副作用證據。

P0	V1-001-003	未知指令當 read、run_tests 拼回 shell 字串、沒有唯一 Policy Middleware。
P1	V1-004-010	政策信任邊界、審計完整性、principal／session／grant、ConfirmMode 與 Tier 混用、escalation 未重新校驗。
P2	V1-011	缺少可重現的真人／Agent 測試手冊，文件完成語意不一致。

- 一版只解一個主要問題，不把 Trust Tier runtime 提前做完。
- P0／P1 關閉前不得進入 v1.0-alpha。
- --yes 只跳過互動，不是認證，也不是 Tier 2。

九個 patch，一版一題

版本	主題	驗收焦點
v0.9.1	Shell containment	未知／混淆指令 fail-closed；argv + shell=False
v0.9.2	Policy Middleware	唯一 dispatcher 與 SecurityContext，無產品旁路
v0.9.3	政策信任邊界	project policy 只能縮權；未知欄位 fail-fast
v0.9.4	Audit integrity	redaction、sequence、控制面保護
v0.9.5	Principal scope	區分 provider key 與授權 principal／scopes
v0.9.6	Session 認證基礎	高權限 step-up；--yes ≠ 認證
v0.9.7	Trust grant store	使用者擁有、工作區外、可 revoke
v0.9.8	Decision matrix	與 ConfirmMode 分離的可測決策表
v0.9.9	Evidence closure	每次 escalation 後重新校驗；attempt 證據閉合

清債序列在 v0.9.9 停止。下一功能動作只能另開 v1.0-alpha 的 Trust Tier change，不得把 runtime 塞回 patch。

後來被寫進禁令的設計原則

Fail-closed	未知、混淆、無法解析的指令不是 read，而是拒絕。缺少 SecurityContext 同樣拒絕，不合成相容上下文。
唯一入口	六個 public tools 與 CrewAI adapter 都走 security.middleware.invoke()。內部 underscore helper 不是產品授權入口。
Restrict-only	council.policy.yaml 是專案擁有的縮權輸入，不能設定 trust tier，也不能用未知欄位換取寬鬆行為。
語意分離	ConfirmMode、--yes、Trust Tier、grant、principal scope 各做一件事。跳過詢問不是授權。
證據閉合	Verification 只使用同一 pipeline attempt 的結構化 tool evidence。Escalation 產生新輸出後必須再校驗。
宣稱克制	分類是 pattern／registry 啟發式，不是完整安全機制。禁止把 command boundary 說成 OS sandbox。

Trust Tier 0／1／2 成為可執行行為

Tier 0 預設 所有需確認操作都要問。適合陌生工作區與高風險變更。	Tier 1 安全自動 被判定為安全的指令可自動執行；其餘仍需確認或拒絕。	Tier 2 全自動 需明確啟用。不是 --yes，也不是專案政策能打開的開關。
--	--	---

- CLI 以 --trust-tier 選擇；決策走 matrix-v2，與 ConfirmMode 分離。
- council trust grant／revoke／list 管理使用者擁有的授權。
- Policy Middleware 串起分類、政策、principal、grant、tier 與審計。
- Alpha 准入要求：v0.9.1–v0.9.9 已發佈、P0／P1 關閉、check.sh 通過。

公開測試：凍結語意，用證據開門

beta.1 · 2026.08.11

- 凍結對外 schema 與安全語意
- 公告固定 beta tag／commit
- 公開測試文件就緒
- Stop-line 文件完成

Agent 測試閘門

- 獨立 Agent 走完公開測試矩陣
- ACCEPTED GATE A
- LIVE-01 PASS（含金鑰重跑）
- 以程式、審計與副作用證據判定

beta.2 · 2026.08.12

- 承接 beta.1 的 Trust Tier 凍結
- 無新的安全語意
- 記錄 check.sh 發版證據
- 目前已發佈基線

仍未關閉

獨立真人 TTY 測試仍待後續 beta。GA 前尚需 audit predecessor hash chain，以及剩餘 Definition of Done。公開測試發現 P0 必須立即停止。

v1.0.0-beta.2 能力一覽

已交付

- 三階段管線與 escalation
- OpenRouter + YAML Preset
- Typer CLI 與 sandbox／audit 子命令
- Filesystem 與 shell tools
- run_tests 結構化報告
- max_tool_calls 與 tool 摘要
- 指令分類與互動確認
- 唯一 Policy Middleware
- 結構化審計 (redaction／sequence)
- restrict-only 專案政策
- Trust Tier 0／1／2 runtime
- matrix-v2 決策表
- Verification attempt 證據閉合
- Agent 公開測試 GATE A + LIVE-01

未完／不做滿

- Audit predecessor hash chain
- 獨立真人 TTY 測試
- OS／容器級 sandbox
- GA 剩餘 DoD 與文件最終對齊

必須寫清楚的邊界

不是 OS sandbox	run_command 是受限 simple grammar + argv + shell=False。這是 command boundary，擋不住 host 使用者、外部程序，或 run_tests 拉起的專案程式碼。
政策不能提權	council.policy.yaml 只能加限制，不能設定 Trust Tier。專案檔不是使用者授權來源。
--yes 不是 Tier 2	它只跳過互動確認，供 CI 使用。無 TTY 且未加 --yes 時，寫入／危險操作會被拒絕。
審計尚未錨定	現行是 per-event integrity substrate：有 redaction、sequence、canonical ID。沒有 predecessor-linked、也沒有外部錨點的 hash chain。
WorkspaceGuard 的範圍	filesystem 驗證路徑；shell 驗證 cwd 與受支援命令的路徑運算元。symlink 與穿越有測，但不是核心隔離。
使用前提	只在信任、可丟棄、無真實秘密的工作區使用。發現 P0 立即停止公開測試。

模組地圖：能力與禁令同頁

cli.py	Typer 入口。禁止直接呼叫 tools/*。
orchestrator.py	三階段協調；安裝 SecurityContext、確認政策、審計與專案政策。
crews/	Planning／Execution／Verification。Execution 掛載 tools；wrapper 只轉接，不做授權。
tools/	純 Python。回傳 ToolResult。禁止 import CrewAI。可預期錯誤禁止 throw。
sandbox/	WorkspaceGuard 與 session。路徑 denylist 與政策聯集。
security/	classifier、confirm、policy、middleware、trust、audit。產品路徑的唯一決策層。

這張圖要讀的不是目錄，而是邊界：CLI 不管工具、tools 不管框架、middleware 不管「方便起見的相容上下文」。禁令是架構的一部分。

工程契約：怎麼合併，怎麼發版



版本號兩處一致 pyproject.toml 與 src/council_agent/__init__.py。禁止在 feature/* 做 chore: release 或打 tag。

Definition of Done tasks 全勾、delta 已 sync、change 已 archive、./scripts/check.sh 全過。

測試硬性規範 檔案 I/O 用 tmp_path；shell 用跨平台指令；.council/sessions/ 不進 git；舊測試零修改仍須全過。

用數字回看這段路

21

發行標籤

v0.1.0 到 v1.0.0-beta.2

282

提交次數

main 線上的歷史長度

9

清債 patch

v0.9.1 到 v0.9.9，一版一題

4

管線階段

計劃、執行、校驗、升級

時間很短，版本卻很密。這是刻意的：Tool-First 用小步把能力送出去；清債序列用更小的步把風險從 DoD 裡拔掉。密度來自「不一次做完多層整合」，不是來自趕工。

規格能力目前分成 orchestration、tools、sandbox、security、release-prep。程式與規格共存，delta 優先於重寫。

通往 GA 的剩餘工作

01	Audit hash chain	predecessor-linked、可驗證、防事後篡改；必要時再談外部錨點。
02	獨立真人 TTY	無前置脈絡的真人走完手冊。Agent 閘門已過，不能代替真人確認。
03	DoD 收斂	Trust grant／高權限解鎖／威脅模型文件等 ROADMAP 仍未勾選的項目，以證據關閉。
04	宣稱對齊	README、docs/index、ROADMAP 現況必須與套件行為同一版語意，避免過期入口。

GA 的標準不是「功能看起來齊了」，而是：無 active change、P0 全關、審計可證明未被改寫、真人與 Agent 測試都走得通。

結語

先讓 Agent 動手， 再讓系統拒絕不該做的事。

規格先於程式。證據先於宣稱。一版一題。能力可以早交付，信任必須可測試。

■ ■ ■ Council Agent • v1.0.0-beta.2 • 2026.08.12